Deep Reinforcement Learning

Actor-Critic Algorithms

Prof. Dr. Sebastian Peitz

Scientific Machine Learning group @ Paderborn University

Winter term 2026/27

PADERBORN UNIVERSITY

Content

Content

- Two alternative derivations of the policy gradient theorem
  - Bottom-up derivation via the Q-function
  - Sampling with reduced variance
- Advantage functions
- Design decisions
- Off-policy actor-critic
- Critics as baselines

Where are we?

Chapter	Topic	Content
	Basics & tabular methods	
1-5	Bandits, MDPs, Dynamic Programming, Monte Carlo, TD Learning	RL basics in finite dimensions
	Deep-learning-based methods	
6	Brief introduction to deep learning	The basics for what comes next
7	Value function approximation	Value estimation with function approximation
8	Deep Q-learning	Q-learning with neural networks
9	Policy gradients	Direct optimization of the policy
10	Actor-critic algorithms	Improved policy gradients via value functions
11	Advanced algorithms (Part I): From policy gradient to PPO	
12	Advanced algorithms (Part II): From Q-learning to Soft Actor-Critic	
13	Exploration	
	Model-Based Control	
	Advanced Topics	

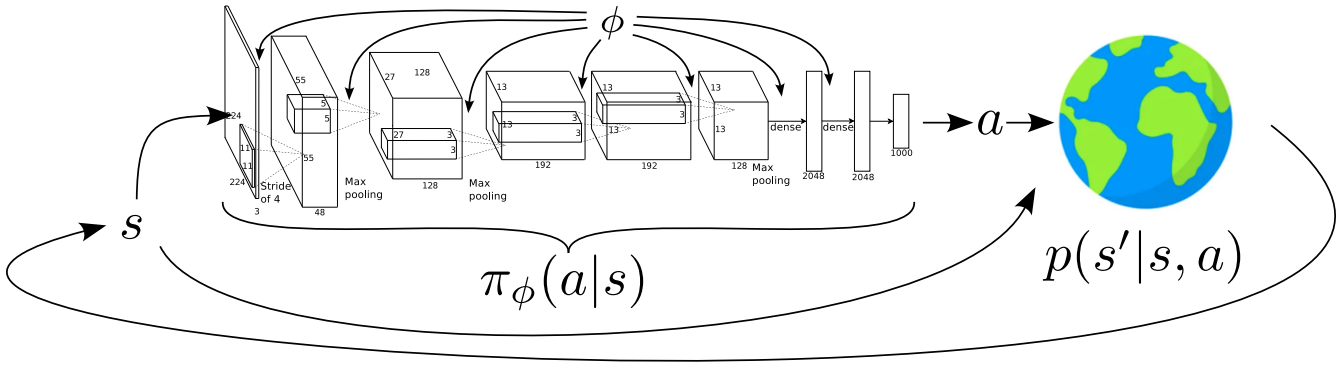
Lecture contents

A more detailed derivation of the policy gradient

The Q-function version of the policy gradient

- Recall that we started with the objective of maximizing the value directly by optimizing over our policy network weights:

$$\begin{aligned}\phi^* &= \arg\max_{\phi} \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\sum_{t=0}^{T-1} r_t \right] \\ &= \arg\max_{\phi} \mathbb{E}_{s_0 \sim p_0} \left[V^{\pi_\phi}(s_0) \right].\end{aligned}$$



Inspired by Sergey Levine's CS285 lecture.

- Using the definition of the expectation of continuous random variables and the $\log$ derivative trick, we arrived at a formulation of the policy gradient from which we can sample using trajectory data ($\tau = (s_0, a_0, \ldots, s_{T-1}, a_{T-1}, s_T)$):

$$\nabla_\phi L_\pi(\phi) = \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\left(\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_t|s_t) \right) \left(\sum_{t=0}^{T-1} r_t \right) \right] \approx \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) \right) \left(\sum_{t=0}^{T-1} r_{i,t} \right).$$

- **Goal**: derive the policy gradient a second time, but this time using the Q-function instead of the return $\sum_{t=0}^{T-1} r_{i,t}$.
- **Starting point**: the RL objective and its gradient:

$$L_\pi(\phi) = \mathbb{E}_{s_0 \sim p_0} \left[V^{\pi_\phi}(s_0) \right] \qquad \Rightarrow \qquad \nabla_\phi L_\pi(\phi) = \mathbb{E}_{s_0 \sim p_0} \left[\nabla_\phi V^{\pi_\phi}(s_0) \right] = \int_{\mathcal{S}} p_0(s) \nabla_\phi V^{\pi_\phi}(s) \, \mathrm{d}s. \tag{1}$$

Deriving the policy gradient (1)

- To derive the policy gradient, we need to take the **gradient of the value function**, i.e., $\nabla_\phi V^{\pi_\phi}(s)$.
- Our **goal is to introduce the Q-function** (we will see the reason for this later). What's the relation between the Q-function and the value function for continuous actions?

$$\underbrace{V^{\pi_\phi}(s) = \sum_{a \in \mathcal{A}} \pi_\phi(a|s) Q^{\pi_\phi}(s,a)}_{\text{finite } \mathcal{A}} \qquad \text{vs.} \qquad \underbrace{V^{\pi_\phi}(s) = \int_{\mathcal{A}} \pi_\phi(a|s) Q^{\pi_\phi}(s,a)\, \mathrm{d}a}_{\text{continuous } \mathcal{A}}.$$

- Now let's **take the derivative**! Both π and Q^{π_ϕ} depend on $\phi \Rightarrow$ product rule ($\frac{d}{dx}(f(x) \cdot g(x)) = f(x)\frac{dg}{dx} + \frac{df}{dx}g(x)$):

$$\begin{aligned}
\nabla_\phi V^{\pi_\phi}(s) &= \nabla_\phi \left(\int_{\mathcal{A}} \pi_\phi(a|s) Q^{\pi_\phi}(s,a)\, \mathrm{d}a \right) = \int_{\mathcal{A}} \nabla_\phi \left[\pi_\phi(a|s) Q^{\pi_\phi}(s,a) \right] \mathrm{d}a = \int_{\mathcal{A}} \nabla_\phi \pi_\phi(a|s) Q^{\pi_\phi}(s,a) + \pi_\phi(a|s) \nabla_\phi Q^{\pi_\phi}(s,a)\, \mathrm{d}a \\
&= \underbrace{\int_{\mathcal{A}} \nabla_\phi \pi_\phi(a|s) Q^{\pi_\phi}(s,a)\, \mathrm{d}a}_{=\xi(s)} + \int_{\mathcal{A}} \pi_\phi(a|s) \nabla_\phi Q^{\pi_\phi}(s,a)\, \mathrm{d}a
\end{aligned}$$

- In total, we obtain the following **expression for $\nabla_\phi V^{\pi_\phi}(s)$**:

$$\nabla_\phi V^{\pi_\phi}(s) = \xi(s) + \int_{\mathcal{A}} \pi_\phi(a|s) \nabla_\phi Q^{\pi_\phi}(s,a)\, \mathrm{d}a. \tag{2}$$

Deriving the policy gradient (2)

- Before we continue with (2), let's study $Q^{\pi_\phi}(s,a)$ in some more detail, with the **goal to find a Bellman recursion** formula:

$$Q^{\pi_\phi}(s,a) = r + \int_{\mathcal{S}} p\left(s'|s,a\right) V^{\pi_\phi}(s')\,\mathrm{d}s'. \tag{3}$$

- Since neither r nor $p\left(s'|s,a\right)$ depend on ϕ, taking the gradient of (3) yields

$$\nabla_\phi Q^{\pi_\phi}(s,a) = \int_{\mathcal{S}} p\left(s'|s,a\right) \nabla_\phi V^{\pi_\phi}(s')\,\mathrm{d}s'. \tag{4}$$

- We can substitute this expression into (2) and swap integrals due to linearity:

$$\nabla_\phi V^{\pi_\phi}(s) = \xi(s) + \int_{\mathcal{A}} \pi_\phi(a|s) \underbrace{\left[\int_{\mathcal{S}} p\left(s'|s,a\right) \nabla_\phi V^{\pi_\phi}(s')\,\mathrm{d}s'\right]}_{(4)} \mathrm{d}a = \xi(s) + \int_{\mathcal{S}} \underbrace{\left[\int_{\mathcal{A}} \pi_\phi(a|s) p\left(s'|s,a\right)\,\mathrm{d}a\right]}_{=p(s\to s'|1,\pi_\phi)} \nabla_\phi V^{\pi_\phi}(s')\,\mathrm{d}s'.$$

- Here, $p\left(s \to s'|1,\pi_\phi\right)$ denotes the probability of moving from s to s' in exactly one step, given the policy π_ϕ:

$$\nabla_\phi V^{\pi_\phi}(s) = \xi(s) + \int_{\mathcal{S}} p\left(s \to s'|1,\pi_\phi\right) \nabla_\phi V^{\pi_\phi}(s')\,\mathrm{d}s'. \tag{5}$$

Deriving the policy gradient (3)

- Equation (5) is a Bellman recursion equation, and we can insert the same expression at s' on the right:

$$\nabla_\phi V^{\pi_\phi}(s) = \xi(s) + \int_{\mathcal{S}} p\left(s \to s' | 1, \pi_\phi\right) \underbrace{\left[\xi(s') + \int_{\mathcal{S}} p\left(s' \to s'' | 1, \pi_\phi\right) \nabla_\phi V^{\pi_\phi}(s'') \, \mathrm{d}s''\right]}_{= \nabla_\phi V^{\pi_\phi}(s')} \mathrm{d}s'.$$

- We can unroll this expression for an entire trajectory τ of length T. After some resorting, we obtain:

$$\nabla_\phi V^{\pi_\phi}(s) = \xi(s) + \int_{\mathcal{S}} p\left(s \to s' | 1, \pi_\phi\right) \xi(s') \, \mathrm{d}s' + \int_{\mathcal{S}} p\left(s \to s'' | 2, \pi_\phi\right) \xi(s'') \, \mathrm{d}s'' + \dots$$

- We can write this cleanly as a sum over all future timesteps t:

$$\nabla_\phi V^{\pi_\phi}(s) = \int_{\mathcal{S}} \sum_{t=0}^{T-1} p\left(s \to x | t, \pi_\phi\right) \xi(x) \, \mathrm{d}x. \tag{6}$$

  Note: the previously separate case $\xi(s)$ is included, i.e., $\int_{\mathcal{S}} p\left(s \to s | 0, \pi_\phi\right) \xi(x) \, \mathrm{d}x = \xi(s)$.

Deriving the policy gradient (4)

- We can now insert (6) into the gradient of our RL objective (Eq. (1)):

$$\nabla_\phi L_\pi(\phi) = \mathbb{E}_{s_0 \sim p_0}\left[\nabla_\phi V^{\pi_\phi}(s_0)\right] = \int_{\mathcal{S}} p_0(s) \nabla_\phi V^{\pi_\phi}(s)\, \mathrm{d}s = \int_{\mathcal{S}} p_0(s) \left[\int_{\mathcal{S}} \sum_{t=0}^{T-1} p\left(s \to x | t, \pi_\phi\right) \xi(x)\, \mathrm{d}x\right] \mathrm{d}s.$$

- By changing the order of integration, we isolate the total expected time spent in state x:

$$\nabla_\phi L_\pi(\phi) = \int_{\mathcal{S}} \left[\int_{\mathcal{S}} p_0(s) \sum_{t=0}^{T-1} p\left(s \to x | t, \pi_\phi\right) \mathrm{d}s\right] \xi(x)\, \mathrm{d}x.$$

- The term inside the brackets is the **undiscounted state visitation frequency** (or the expected number of times state x is visited in an episode), which we denote as $\eta_\phi(s)$:

$$\eta_\phi(s) = \sum_{t=0}^{T-1} p\left(s_t = s | \pi_\phi\right). \tag{7}$$

- This simplifies our expression to:

$$\nabla_\phi L_\pi(\phi) = \int_{\mathcal{S}} \eta_\phi(s) \xi(s)\, \mathrm{d}s.$$

Deriving the policy gradient (5)

- Reintroducing the full definition of $\xi(s) = \int_{\mathcal{A}} \nabla_\phi \pi_\phi(a|s) Q^{\pi_\phi}(s,a)\,\mathrm{d}a$ (and keeping s as our state variable):

$$\nabla_\phi L_\pi(\phi) = \int_{\mathcal{S}} \eta_\phi(s) \int_{\mathcal{A}} \nabla_\phi \pi_\phi(a|s) Q^{\pi_\phi}(s,a)\,\mathrm{d}a\,\mathrm{d}s.$$

- Apply the $\log$-derivative identity:

$$\nabla_\phi L_\pi(\phi) = \int_{\mathcal{S}} \eta_\phi(s) \int_{\mathcal{A}} \pi_\phi(a|s) \nabla_\phi \log \pi_\phi(a|s) Q^{\pi_\phi}(s,a)\,\mathrm{d}a\,\mathrm{d}s.$$

- In conclusion, the gradient can be expressed using the unnormalized state visitation measure $\eta_\phi(s)$:

$$\nabla_\phi L_\pi(\phi) = \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_t|s_t) Q^{\pi_\phi}(s_t,a_t) \right]. \quad (8)$$

Expectations in continuous spaces

The expected value of a function $f(s)$ is

$$\mathbb{E}_{s\sim p}\left[f(s)\right] = \int p(s) f(s)\,\mathrm{d}s.$$

Here, p is the density according to which s is distributed, with $\int p(s)\,\mathrm{d}s = 1$.

A convenient identity

$$\nabla_\phi \pi_\phi(a|s) = \pi_\phi(a|s) \nabla_\phi \log \pi_\phi(a|s)$$

Note: In the infinite-horizon case, (7) becomes $\eta_\phi(s) = \sum_{t=0}^{\infty} p\left(s_t = s | p_0, \pi_\phi\right) \Rightarrow$ Equation (8) becomes

$$\nabla_\phi L_\pi(\phi) = \mathbb{E}_{s\sim\eta_\phi, a\sim\pi_\phi}\left[\nabla_\phi \log \pi_\phi(a|s) Q^{\pi_\phi}(s,a)\right].$$

Comparison of the two formulations

Here's the policy gradient theorem in the two versions we have derived (Sampling versions in blue).

Policy gradient theorem – formulation via reward trajectories

$$\nabla_\phi L_\pi(\phi) = \mathbb{E}_{\tau\sim p_\phi(\tau)}\left[\sum_{t=0}^{T-1}\nabla_\phi \log \pi_\phi(a_t|s_t)\left(\sum_{t'=t}^{T-1} r_{t'}\right)\right] \approx \frac{1}{N}\sum_{i=1}^{N}\left(\sum_{t'=t}^{T-1}\nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t})\left(\sum_{t'=t}^{T-1} r_{i,t'}\right)\right).$$

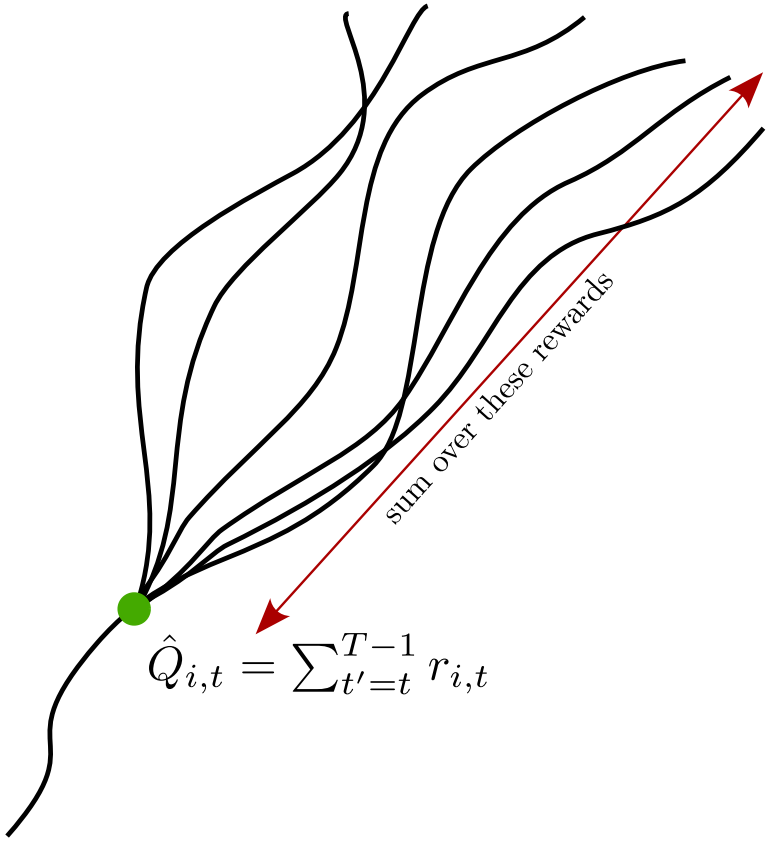
Policy gradient theorem – formulation using the Q-function

$$\nabla_\phi L_\pi(\phi) = \mathbb{E}_{\tau\sim p_\phi(\tau)}\left[\sum_{t=0}^{T-1}\nabla_\phi \log \pi_\phi(a_t|s_t) Q^{\pi_\phi}(s_t,a_t)\right] \approx \frac{1}{N}\sum_{i=1}^{N}\left(\sum_{t=0}^{T-1}\nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) Q^{\pi_\phi}(s_{i,t},a_{i,t})\right).$$

Questions:

1. Which formulation is more accurate? ⇒ The second version has smaller variance! (next slide)
2. Which type of sampling can we use for the formulations?
   ⇒ First one: Monte Carlo sampling.
   ⇒ Second: Temporal Difference learning.
3. How can we even sample from Q^{π_ϕ} if it is not explicitly defined? ⇒ We'll see soon ("*critic*").

A much simpler way to arrive at this formulation



- Observe that the **reward-to-go** $\hat{Q}_{i,t} = \sum_{t'=t}^{T-1} r_{i,t'}$ is an **unbiased estimate** of $Q(s_t, a_t)$.
- But: $\hat{Q}_{i,t}$ is a **single-sample estimate** of the reward-to-go.
- To reduce the variance, it would be a lot better to simply use the **true expected reward-to-go**:

$$\sum_{t'=t}^{T-1} \mathbb{E}_{\pi_\phi}\left[r_t | s_t, a_t\right] = Q^{\pi_\phi}(s_{i,t}, a_{i,t}).$$

- Replacing $\sum_{t'=t}^{T-1} r_{i,t'}$ by $Q^{\pi}(s_{i,t}, a_{i,t})$ yields

$$\nabla_\phi L_\pi(\phi) = \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_t|s_t) Q^\pi(s_t, a_t) \right].$$

Note: The inconsistency between the supersripts π and π_ϕ for Q is not accidental. We will soon see what's the reason behind this.

Inspired by Sergey Levine's CS285 lecture.

What about the baseline?

We would like to reduce the variance of the new formulation using a baseline b:

$$\nabla_\phi L_\pi(\phi) = \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_t|s_t)[Q^\pi(s_t, a_t) - b] \right] \approx \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t})[Q^\pi(s_{i,t}, a_{i,t}) - b] \right).$$

How do we choose b?

- A natural choice (analogous to the baseline for version one) might be the average the Q^π value over our samples:

$$b = \frac{1}{N} Q^\pi(s_{i,t}, a_{i,t}).$$

- Unfortunately, an action-dependent average leads to a biased policy gradient (i.e., leads to the wrong gradient) 😩

An alternative (and even lower-variance) approach:

- Average over **all the possibilities starting in the state s_t** (not just in the time step t)!
- How do we do this?

$$b = \mathbb{E}_{a_t \sim \pi(\cdot|s_t)} \left[Q^\pi(s_t, a_t) \right] = V^\pi(s_t).$$

Advantage functions

A very good baseline for policy gradients

- If we're using the value function $V^\pi(s_t)$ as our baseline, we obtain the following expression:

$$\nabla_\phi L_\pi(\phi) = \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_t|s_t) [Q^\pi(s_t, a_t) - V^\pi(s_t)] \right] \approx \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) [Q^\pi(s_{i,t}, a_{i,t}) - V^\pi(s_{i,t})] \right).$$

- This one has a very intuitive interpretation:
  - If $Q^\pi(s, a) - V^\pi(s) > 0$, then a is **better than the average action** according to our current policy π.
  - If $Q^\pi(s, a) - V^\pi(s) < 0$, then a is **worse than the average action**.

Policy gradient with advantage function

The *advantage function* $A^\pi(s, a)$ describes how much better the action a is over the average action when following π:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

Policy gradient with value baseline: "maximize the policy likelihood, weighted by the advantage function":

$$\nabla_\phi L_\pi(\phi) = \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_t|s_t) A^\pi(s_t, a_t) \right] \approx \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) A^\pi(s_{i,t}, a_{i,t}) \right).$$

The actor-critic framework

- Remember the inconsistency (supersripts π and π_ϕ for Q) in our "simple derivation" of the policy gradient formulation,

$$\nabla_\phi L_\pi(\phi) = \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_t|s_t) Q^\pi(s_t, a_t) \right] ?$$

The actor-critic framework

Actor: Neural network with parameters ϕ approximates the policy:

$$\pi_\phi \approx \pi.$$

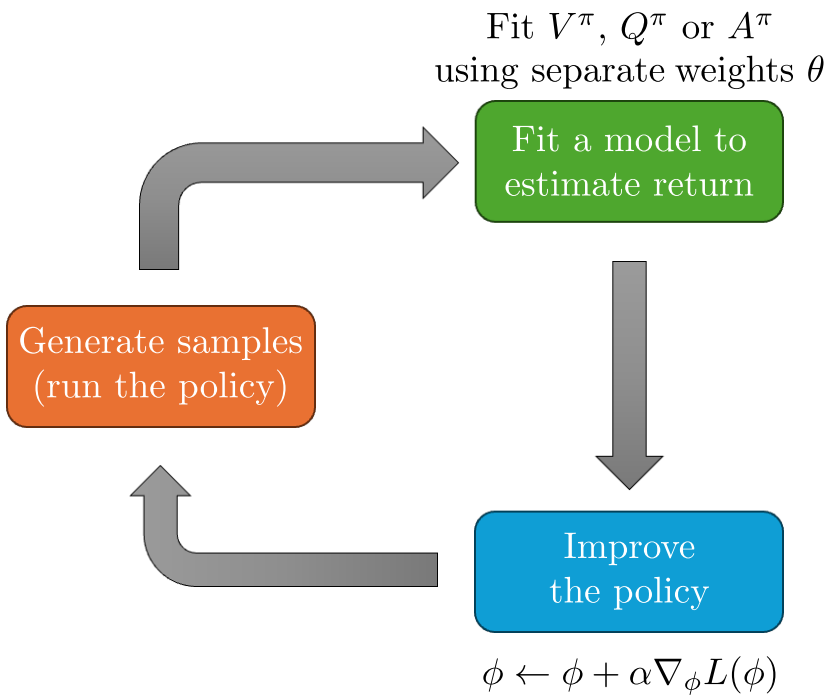
Critic: Approximates V^π / Q^π / A^π and *criticizes* the actor:

$$V_\theta \approx V^\pi, \qquad Q_\theta \approx Q^\pi, \qquad A_\theta \approx A^\pi.$$

Policy gradient using, e.g., Q_θ:

$$\nabla_\phi L_\pi(\phi) = \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_t|s_t) Q_\theta(s_t, a_t) \right].$$

- As well as Question 3. when we discussed the difference between the r-based and the Q-based versions of the policy gradient: "How can we even sample from Q^{π_ϕ} if it is not explicitly defined"?



Inspired by Sergey Levine's CS285 lecture.

An actor-critic algorithm (1)

- Now consider the baseline version where we use the advantage function A^π to weight the log gradient update:

$$\nabla_\phi L_\pi(\phi) = \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_t|s_t) A^\pi(s_t, a_t) \right] \approx \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) A^\pi(s_{i,t}, a_{i,t}) \right).$$

- **Key questions**:
  - What should we fit? V^π, Q^π or A^π?
  - To which target should we fit?
- Recall the definition of the Q-function (with $\gamma = 1$):

$$Q^\pi(s_t, a_t) = \mathbb{E}_\pi \left[r_t + Q^\pi(s_{t+1}, a_{t+1}) | s_t, a_t \right] = \mathbb{E}_\pi \left[r_t | s_t, a_t \right] + \mathbb{E}_\pi \left[\sum_{t'=t+1}^{T-1} r_{t'} \middle| s_t, a_t \right] = \mathbb{E}_\pi \left[r_t | s_t, a_t \right] + \underbrace{\sum_{t'=t+1}^{T-1} \mathbb{E}_\pi [r_{t'} | s_t, a_t]}_{= V^\pi(s_{t+1})}$$

$$= \mathbb{E}_\pi \left[r_t | s_t, a_t \right] + \mathbb{E}_{s_{t+1} \sim p(\cdot|s_t, a_t)} \left[V^\pi(s_{t+1}) \right].$$

- Next, we're going to make *two assumptions* to turn this into an algorithm in which we can use experience.

An actor-critic algorithm (2)

Two assumptions that lead to the following approximation:

$$Q^\pi(s_t, a_t) = \mathbb{E}_\pi\left[r_t | s_t, a_t\right] + \mathbb{E}_{s_{t+1} \sim p(\cdot|s_t, a_t)}\left[V^\pi(s_{t+1})\right] \approx r_t + V^\pi(s_{t+1}).$$

1. We just **take the next reward instead of considering the expectation**:

   $$r_t \approx \mathbb{E}_\pi\left[r_t | s_t, a_t\right].$$

   - This is often very reasonable.
   - If the rewards depends deterministically on the state s_t and the action a_t, then this isn't even an approximation.

   💡 In this case, people often use the notation $r(s, a)$ to denote that the reward is a deterministic function.

2. Instead of considering the expectation over all next possible states, we **take the value of the next state we see in our executed trajectory as a representative**:

   $$\mathbb{E}_{s_{t+1} \sim p(\cdot|s_t, a_t)}\left[V^\pi(s_{t+1})\right] \approx V^\pi(s_{t+1}).$$

   - This is an assumption that introduces bias.
   - But it is often still a very reasonable assumption.
   - The strong reduction in variance justifies such a biased (but simple-to-assess) estimator.

An actor-critic algorithm (3)

- Now that we have our approximation $Q^\pi(s_t, a_t) \approx r_t + V^\pi(s_{t+1})$, what to do with it?
- Let's take another look at the *advantage function* A^π:

$$A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t) \approx r_t + V^\pi(s_{t+1}) - V^\pi(s_t) = \hat{A}^\pi(s_t, a_t).$$

- So let's just approximate $V^\pi \approx V_\theta$!
- How do we do this? $\Rightarrow$ We have seen this already.
  - Monte Carlo estimates from entire trajectories:

$$L_V(\theta) = \sum_{k=1}^{N} \Big(g_k - V_\theta(s_k)\Big)^2 \quad \Rightarrow \quad \theta \leftarrow \theta + \alpha[g - V_\theta(s)]\nabla_\theta V_\theta(s).$$

  - TD estimates from single-sample transitions using semi-gradients:

$$L_V(\theta) = \sum_{k=1}^{N} \Big(r_k + V_\theta(s_{k+1}) - V_\theta(s_k)\Big)^2$$

$$\Rightarrow \quad \theta \leftarrow \theta + \alpha\big[r + V_\theta(s') - V_\theta(s)\big]\nabla_\theta V_\theta(s).$$

Algorithm: Batch actor-critic

1. Sample $\{s_i, a_i, s_i'\}_{i=1}^N$ using $\pi_\phi(a|s)$.
2. Fit $V_\theta(s)$ to the sampled rewards.
3. Compute advantage:

$$A_\theta(s_i, a_i) = r_i + V_\theta(s_i') - V_\theta(s_i).$$

4. Gradient:

$$\nabla_\phi L_\pi(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\phi \log \pi_\phi(a_i|s_i) A_\theta(s_i, a_i).$$

5. Gradient ascent: $\phi \leftarrow \phi + \alpha \nabla_\phi L_\pi(\phi)$.

💡 We already know what's wrong with this approach!
$\Rightarrow$ The target changes along with the fitted value function in step 2.
$\Rightarrow$ Use target network $\bar{\theta}$?

Re-introducing the discount factor

Re-introducing the discount factor (1)

- For “simplicity”, we have disregarded the discount factor γ for now (i.e., $\gamma = 1$).
- However, the derivations can be done with a discount factor in a very similar fashion.
- The changes we observe occur in two places:
  1. The obvious one: Our Q-function is now the discounted version:

$$Q^{\pi}(s,a) = \mathbb{E}_{\pi}\left[r + \gamma Q^{\pi}(s',a')|s,a\right].$$

  💡 The same obviously holds for V and A.

  2. The subtle one: The state distribution (“state visitation probability”) changes:

$$\eta_{\phi}(s) = \sum_{t=0}^{T-1} \gamma^t p\left(s_t = s|\pi_{\phi}\right).$$

  💡 The proof is quite technical and requires swapping the sum over the time steps t and the integration over $\mathcal{S}$ in the policy gradient derivation.

- When sampling, point 2. is taken care of automatically, as we will sample according to this new distribution automatically. We thus obtain the same formulation (here using A^{π}):

$$\nabla_{\phi} L_{\pi}(\phi) = \mathbb{E}_{\tau \sim p_{\phi}(\tau)}\left[\sum_{t=0}^{T-1} \nabla_{\phi} \log \pi_{\phi}(a_t|s_t) A^{\pi}(s_t,a_t)\right] \approx \frac{1}{N}\sum_{i=1}^{N}\left(\sum_{t=0}^{T-1} \nabla_{\phi} \log \pi_{\phi}(a_{i,t}|s_{i,t}) A^{\pi}(s_{i,t},a_{i,t})\right),$$

  where point 1. is “hidden” in A^{π} and point 2. is “hidden” in the distribution $\tau \sim p_{\phi}(\tau)$.

Re-introducing the discount factor (2)

- For a better understanding, let's make an ad-hoc introduction of discount factors and see where this leads us.
- We start with version one of the policy gradient, i.e., Monte Carlo sampling of rewards using causality:

$$\nabla_\phi L_\pi(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) \left(\sum_{t'=t}^{T-1} \gamma^{t'-t} r_{i,t'} \right) \right). \tag{9}$$

- Alternatively, we can start with the version without considering causality:

$$\nabla_\phi L_\pi(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) \right) \left(\sum_{t'=0}^{T-1} \gamma^{t'} r_{i,t'} \right). \tag{10}$$

⚡ Which one is right? There clearly is a different discount for the rewards in (9) and (10)!

- Let's reformulate (10) and introduce causality again:

$$\begin{aligned} \nabla_\phi L_\pi(\phi) &\approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) \left(\sum_{t'=0}^{T-1} \gamma^{t'} r_{i,t'} \right) \\ &= \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T-1} \gamma^{t} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) \left(\sum_{t'=t}^{T-1} \gamma^{t'-t} r_{i,t'} \right) \end{aligned} \tag{11}$$

Re-introducing the discount factor (3)

$$\underbrace{\nabla_\phi L_\pi(\phi) \approx \frac{1}{N}\sum_{i=1}^{N}\left(\sum_{t=0}^{T-1}\nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t})\left(\sum_{t'=t}^{T-1}\gamma^{t'-t} r_{i,t'}\right)\right)}_{\text{Option 1: (9)}} \quad \text{vs.} \quad \underbrace{\nabla_\phi L_\pi(\phi) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{T-1}\gamma^{t}\nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t})\left(\sum_{t'=t}^{T-1}\gamma^{t'-t} r_{i,t'}\right)}_{\text{Option 2: (11)}}$$

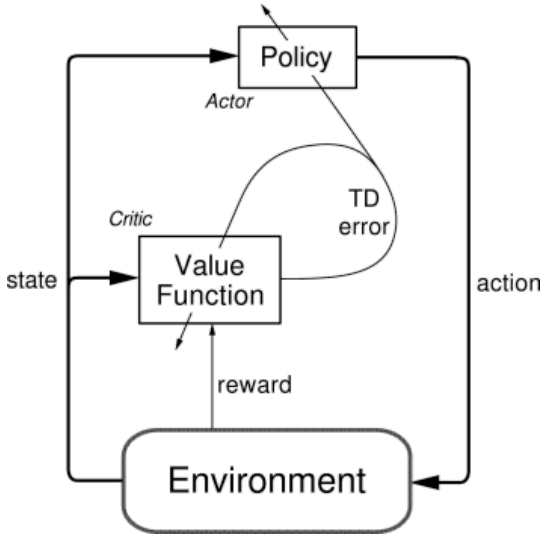
- Option 2 (Equation (11)) is mathematically correct.
- This makes a lot of sense. Since later rewards are less important due to the discount, later actions should also matter less!
- But: **In practice, we use Option 1** (Equation (9))
  - In many tasks, we do care about the long-term behavior.
  - Additional discounting of the policy often leads to deterioration of long-term performance (e.g., in infinite-horizon problems).

Common versions of the policy gradient with discount γ

MC reward sampling:
$$\nabla_\phi L_\pi(\phi) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{T-1}\nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t})\left(\sum_{t'=t}^{T-1}\gamma^{t'-t} r_{i,t'}\right)$$

Advantage function:
$$\nabla_\phi L_\pi(\phi) \approx \frac{1}{N}\sum_{i=1}^{N}\left(\sum_{t=0}^{T-1}\nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t})\underbrace{\left(r_{i,t}+\gamma V_\theta(s_{i,t+1})-V_\theta(s_{i,t})\right)}_{A_\theta(s_{i,t},a_{i,t})}\right)$$

Batch and online AC with discount



Actor-critic framework [Source].

Algorithm: Batch actor-critic with discount

1. Sample $\{s_i, a_i, s'_i\}_{i=1}^N$ using $\pi_\phi(a|s)$.
2. Fit $V_\theta(s)$ to the sampled reward sums.
3. Compute advantages:
$$A_\theta(s_i, a_i) = r_i + \gamma V_\theta(s'_i) - V_\theta(s_i).$$
4. Gradient:
$$\nabla_\phi L_\pi(\phi) \approx \frac{1}{N} \sum_{i=1}^N \nabla_\phi \log \pi_\phi(a_i|s_i) A_\theta(s_i, a_i).$$
5. Gradient ascent: $\phi \leftarrow \phi + \alpha \nabla_\phi L_\pi(\phi)$.

Algorithm: Online actor-critic with discount

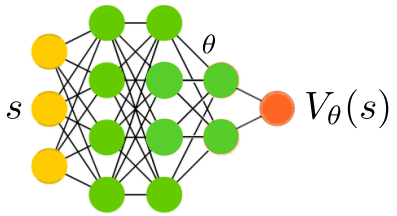
1. Action $a \sim \pi_\phi(a|s) \Rightarrow (s, a, r, s')$.
2. Update $V_\theta(s)$ using the TD error: $\delta = r + \gamma V_\theta(s') - V_\theta(s)$.
3. Compute advantage:
$$A_\theta(s, a) = r + \gamma V_\theta(s') - V_\theta(s).$$
4. Gradient:
$$\nabla_\phi L_\pi(\phi) \approx \nabla_\phi \log \pi_\phi(a|s) A_\theta(s, a).$$
5. Gradient ascent: $\phi \leftarrow \phi + \alpha \nabla_\phi L_\pi(\phi)$.

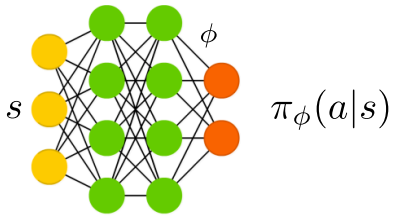
Design decisions

Design decisions

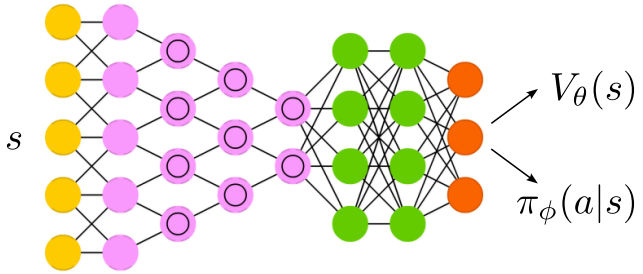
Choice of neural networks

Now that we need to fit two functions, π_ϕ and V_θ, how do we do this in practice?





Two separate networks

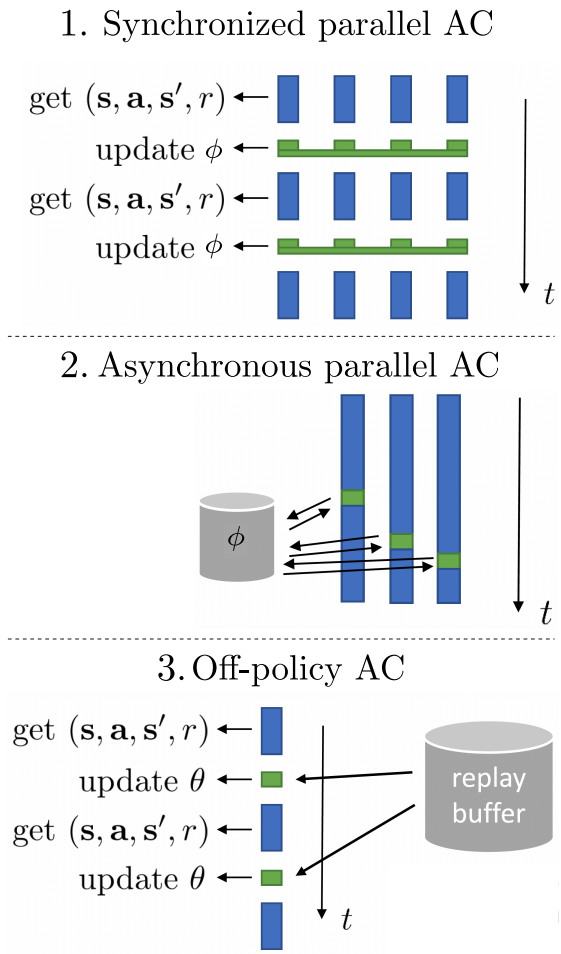


A shared network (e.g., a shared trunk and separate heads)

Inspired by Sergey Levine's CS285 lecture.

Data collection and parallelism

- Online AC works best with batches instead of single samples.
  - Option 1: parallel workers, *synchronous* updates.
  - Option 2: parallel workers, *asynchronous* (but *closely aligned*) updates.
  - Option 3: training from a replay buffer, with data *collected at different times* and *under different policies*.
- The last option requires off-policy training, even though policy gradients are originally on-policy!



Inspired by Sergey Levine's CS285 lecture.

Off-policy actor-critic

Online AC using off-policy data

Algorithm: Off-policy actor-critic with discount

1. Action $a \sim \pi_\phi(a|s) \Rightarrow (s, a, r, s')$ $\Rightarrow$ store in $\mathcal{D}$.
2. Sample batch $\mathcal{B} \subset \mathcal{D}$ (size N) from the buffer.
3. Update $V_\theta(s)$ using the TD error:

$$\min_\theta \frac{1}{N} \sum_{i=1}^{N} \| \underbrace{r_i + \gamma V_\theta(s_i') - V_\theta(s_i)}_{=\delta_i} \|_2^2.$$

4. Compute advantages:

$$A_\theta(s_i, a_i) = r_i + \gamma V_\theta(s_i') - V_\theta(s_i).$$

5. Gradient:

$$\nabla_\phi L_\pi(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\phi \log \pi_\phi(a_i|s_i) A_\theta(s_i, a_i).$$

6. Gradient ascent: $\phi \leftarrow \phi + \alpha \nabla_\phi L_\pi(\phi)$.

The algorithm is broken in two places! Can you spot them?

1. **Step 3**: We are using the wrong target value.
   V_θ is *on-policy* and only valid for the policy π whose data it was trained on!
2. **Step 5**: $\pi_\phi(a|s)$ does not yield the action we would have selected.

$\Rightarrow$ To make actor-critic an off-policy algorithm, we need to fix both!

Online AC: Fixing the value function and policy

Algorithm: Off-policy actor-critic with discount

1. Action $a \sim \pi_\phi(a|s) \Rightarrow (s, a, r, s')$ $\Rightarrow$ store in $\mathcal{D}$.
2. Sample batch $\mathcal{B} \subset \mathcal{D}$ (size N) from the buffer.
3. Update $Q_\theta(s)$ using the TD error:

$$\min_\theta \frac{1}{N} \sum_{i=1}^{N} \| \underbrace{r_i + \gamma Q_\theta(s_i', a_i') - Q_\theta(s_i, a_i)}_{=\delta_i} \|_2^2.$$

4. Compute advantages:

$$A_\theta(s_i, a_i) = r_i + \gamma V_\theta(s_i') - V_\theta(s_i).$$

5. Gradient:

$$\nabla_\phi L_\pi(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\phi \log \pi_\phi\left(a_i^{\pi_\phi} | s_i\right) A_\theta(s_i, a_i).$$

6. Gradient ascent: $\phi \leftarrow \phi + \alpha \nabla_\phi L_\pi(\phi)$.

Step 3: Use the Q-function instead of the value function!

- To approximate the value function of the current policy π_ϕ, we can simply sample $a_i' \sim \pi_\phi(\cdot|s_i')$.
- Recall (for Step 4.): $V^\pi(s_i) = \mathbb{E}_{a_i \sim \pi(\cdot|s_i)}\left[Q(s_i, a_i)\right]$.

Step 5: Sample the actions from the current policy, **not from the replay buffer!**

- $a_i^{\pi_\phi} \sim \pi_\phi(\cdot|s_i)$.

Note 1: The data is still sampled from the "wrong" distribution, i.e., it is not from $p_\phi(s)$!

$\Rightarrow$ There is nothing we can do about this. However, we can see this as positive, as we train a policy on a *broader* distribution.

Note 2: It is very common to use Q_θ instead of A_θ in Step 5, even though skipping the baseline leads to higher variance.

$\Rightarrow$ Skipping the baseline is a good tradeoff, as we can simply sample additional actions (which does **not** require acquiring new states(!)), and thus shrink the variance arbitrarily!

Critics as baselines

Critics as state-dependent baselines

Let's directly compare the policy gradient from the last lecture against our actor-critic procedure from today:

Policy gradient

$$\nabla_\phi L_\pi(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) \left(\left(\sum_{t'=t}^{T-1} \gamma^{t'-t} r_{i,t'} \right) - b \right)$$

+ no bias

− high variance (single-sample estiamte)

Actor-critic

$$\nabla_\phi L_\pi(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) (r_{i,t} + \gamma V_\theta(s_{i,t+1}) - V_\theta(s_{i,t}))$$

+ lower variance (due to critic)

− not unbiased (if critic is imperfect)

Question: Can we use a critic (i.e., V_θ) and still keep the esimator unbiased?

$\Rightarrow$ we can make the baseline b state-dependent (proof very similar to the one from last week)!

$$\nabla_\phi L_\pi(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) \left(\left(\sum_{t'=t}^{T-1} \gamma^{t'-t} r_{i,t'} \right) - V_\theta(s_{i,t}) \right)$$

+ no bias

+ lower variance

Control variates: action dependent baselines

A natural follow-up question is: If a state-dependent baseline is stronger than a constant one, wouldn't a baseline depending on states **and** actions be even better?

$\Rightarrow$ The answer is **yes**! We can take the Q-function as the baseline as well. We call these approaches *control variates*.

(Non-bootstrapped) state-dependent baseline

$$A_\theta(s_t, a_t) = \left(\sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}\right) - V_\theta(s_t)$$

\+ no bias

— higher variance (single-sample)

(Non-bootstrapped) state-action-dependent baseline

$$A_\theta(s_t, a_t) = \left(\sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}\right) - Q_\theta(s_t, a_t)$$

\+ goes to zero in expectation (if critic correct)

— formula is incorrect!

The term that was neglected above:

$$\nabla_\phi L_\pi(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) \left(\hat{Q}_{i,t} - Q_\theta(s_{i,t}, a_{i,t})\right) + \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T-1} \mathbb{E}_{a_t \sim \pi_\theta(\cdot|s_{i,t})} \left[Q_\theta(s_{i,t}, a_t)\right].$$

$\Rightarrow$ This one is often easier to estimate. **Finite $\mathcal{A}$**: compute sum! **Continuous $\mathcal{A}$**: sampling actions is easy!

n-step advantage estimation

Similar to standard TD learning, we can again find some middle ground between the one-step estimate and the MC estimate:

$$\text{One-step TD estimate:} \qquad A_\theta(s_t, a_t) = r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t),$$

$$\text{MC estimate:} \qquad A_\theta(s_t, a_t) = \left(\sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}\right) - V_\theta(s_t).$$

Just as before, we can simulate n steps, before bootstrapping the non-simulated piece of our trajectory:

$$n\text{-step estimate:} \qquad A^n_\theta(s_t, a_t) = \left(\sum_{t'=t}^{t+n} \gamma^{t'-t} r_{t'}\right) + \gamma^n V_\theta(s_{t+n}) - V_\theta(s_t).$$

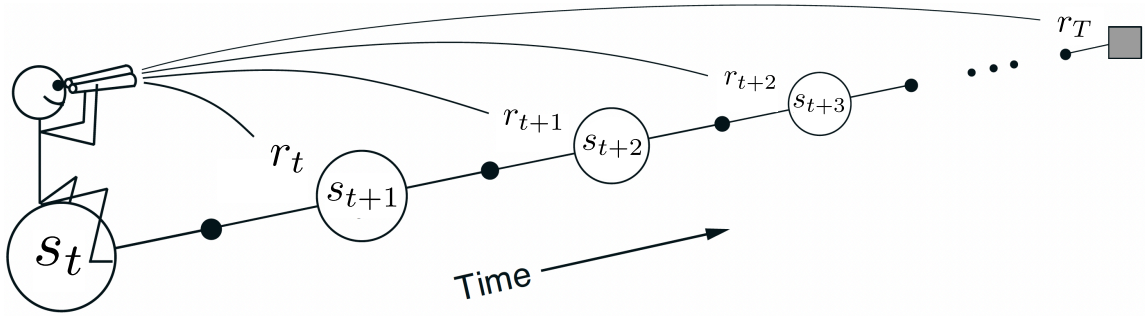
- Near into the future (i.e., $t' \leq t + n$), we often have a small variance, such that a single trajectory gives us a good and unbiased estimate.
- Further into the future (i.e., $t' > t + n$), we likely have a higher variance, such that a single trajectory is not as helpful.
- There, using a bootstrapped estimate of the expected return ($V_\theta(s_{t+n})$) helps us to reduce the variance.
- $n > 1$ often works better, it is a better trade-off between bias and variance.

Generalized advantage estimation

From the n-step estimate, the next step in TD learning was to average over all possible n-step estimates $\Rightarrow$ TD(λ)!

- In the actor-critic setting, this is known as **generalized advantage estimation** (**GAE**):

$$A_\theta^{\text{GAE}}(s_t, a_t) = \sum_n^\infty w_n A_\theta^n(s_t, a_t).$$



The forward view. We decide how to update each state by looking forward to future rewards and states. (Sutton and Barto 1998, Figure 12.4).

- How to weight the individual A_θ^n?
  - put more importance to earlier (low-variance) data.
  - $w_n \propto \lambda^{n-1}$ (exponentiall fall-off).

$$A_\theta^{\text{GAE}}(s_t, a_t) = r_t + \gamma\Big((1-\lambda)V_\theta(s_{t+1}) + \lambda\Big(r_{t+1} + \gamma((1-\lambda)V_\theta(s_{t+2}) + \lambda(r_{t+2} + \ldots))\Big)\Big),$$

$$A_\theta^{\text{GAE}}(s_t, a_t) = \sum_{t'=t}^\infty (\gamma\lambda)^{t'-t}\delta_{t'}, \qquad \text{with TD error} \quad \delta_{t'} = r_{t'} + \gamma V_\theta(s_{t'+1}) - V_\theta(s_{t'}).$$

$\Rightarrow$ The discount factor serves as a means to trade off bias and variance!

References

Sutton, R. S., and A. G. Barto. 1998. *Reinforcement Learning: An Introduction*. MIT Press.